

Financial Market Datasets 2024

Dataset Summary

This dataset provides a 2024 snapshot of global financial, macroeconomic, and geopolitical data for 39 countries. It integrates stock market performance, core economic indicators (GDP, inflation, interest rates), commodities (oil, gold), and risk metrics into a single structured format. Why this dataset was used for the Pawlak DB in Neo4j:

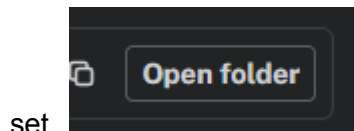
I chose this dataset because its highly interconnected variables are perfectly suited for Neo4j's graph database model. Rather than storing data in isolated, flat tables, this dataset naturally forms a rich network of nodes and relationships:

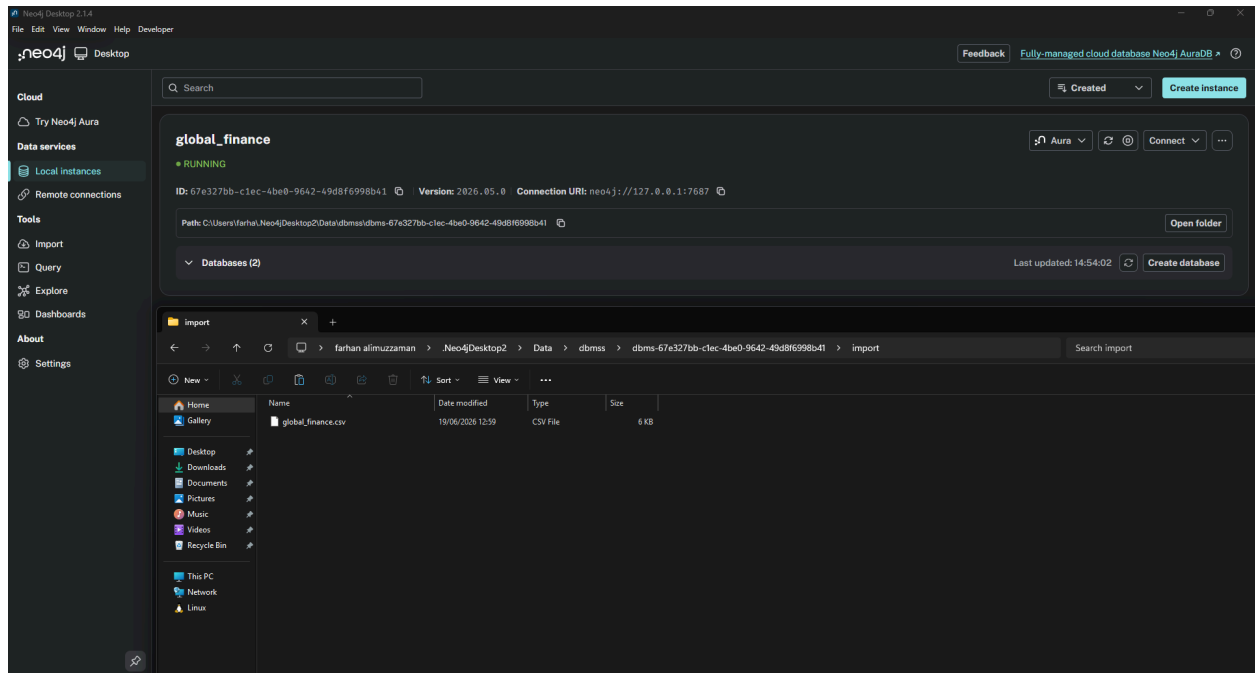
Mapping Dependencies: It allows us to directly link Countries to their Stock Indices, Commodities, and Economic Indicators.

Multi-hop Analysis: Neo4j can easily trace complex, real-world paths—for example, querying how a nation's Political Risk impacts its Credit Rating, which in turn influences its Bond Yields and Market Capitalization.

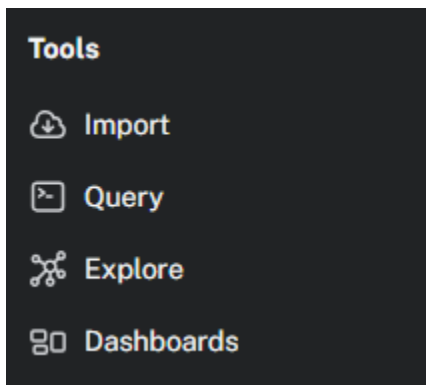
Holistic Modeling: It enables the Pawlak DB to model the global economy as an interconnected web, making it ideal for visualizing global financial dependencies and performing advanced relationship queries that traditional relational databases struggle to handle.

1. First I created an instance in the node4js and imported using the “Open Folder” the data





2. Then i used connect button to connect and go to the tools, Query and we start the query



3. A script that imports a global finance CSV into Neo4j to model a Pawlak Information System by creating country objects, linking them to financial attributes via value

relationships, and organizing them into a two-level credit rating hierarchy.

```
21 // --- 1. uniqueness constraints -----
22 CREATE CONSTRAINT object_id IF NOT EXISTS FOR (o:Object) REQUIRE o.id IS UNIQUE;
23 CREATE CONSTRAINT attr_name IF NOT EXISTS FOR (a:Attribute) REQUIRE a.name IS UNIQUE;
24 CREATE CONSTRAINT group_name IF NOT EXISTS FOR (g:Group) REQUIRE g.name IS UNIQUE;
25
26 // --- 2. core import: objects, attributes, values -----
27 LOAD CSV WITH HEADERS FROM 'file:///global_finance.csv' AS row
28 // 2a. one Object node per country
29 MERGE (o:Object {id: row.Country})
30 // 2b. unwind every column as an (attribute name, value) pair
31 WITH o, row,
32 [
33   {name:'Date', value: row.Date},
34   {name:'Stock_Index', value: row.Stock_Index},
35   {name:'Index_Value', value: row.Index_Value},
36   {name:'Daily_Change_Percent', value: row.Daily_Change_Percent},
37   {name:'Market_Cap_Trillion_USD', value: row.Market_Cap_Trillion_USD},
38   {name:'GDP_Growth_Rate_Percent', value: row.GDP_Growth_Rate_Percent},
39   {name:'Inflation_Rate_Percent', value: row.Inflation_Rate_Percent},
40   {name:'Interest_Rate_Percent', value: row.Interest_Rate_Percent},
41   {name:'Unemployment_Rate_Percent', value: row.Unemployment_Rate_Percent}.
```

✓ // -----

✓ // --- 1. uniqueness constraints -----

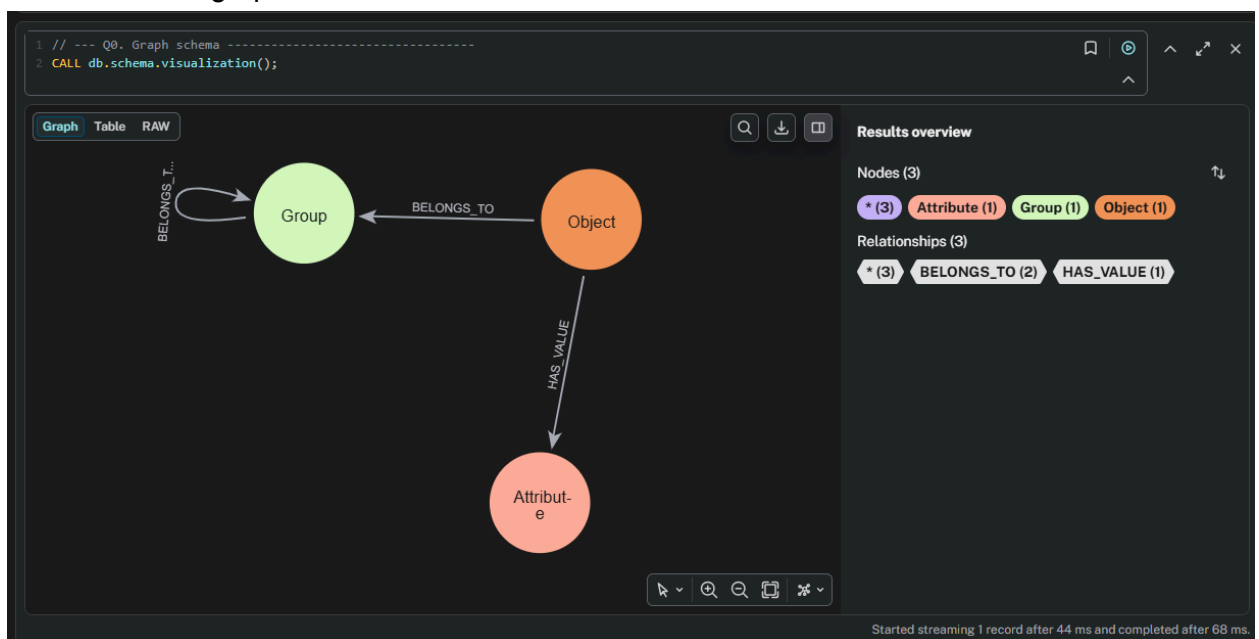
✓ CREATE CONSTRAINT attr_name IF NOT EXISTS FOR (a:Attribute) REQUIRE a.name IS UNIQUE;

✓ CREATE CONSTRAINT group_name IF NOT EXISTS FOR (g:Group) REQUIRE g.name IS UNIQUE;

✓ // --- 2. core import: objects, attributes, values -----

✓ // --- 3. OPTIONAL TASK 3: build group hierarchy from Credit_Rating -----

4. We check if the graph schema works



5. We check the object count (personal message - sorry for being low object count, it was difficult to find a csv with proper data hygiene that can be imported)

```
1 // --- Q1. Count objects |U| -----
2 MATCH (o:Object) RETURN count(o) AS object_count;
```

Table RAW

object_count
39

Started streaming 1 record after 46 ms and completed after 47 ms.

6. Cypher script to check attributes

```
1 // --- Q2. Count attributes |A| -----
2 MATCH (a:Attribute) RETURN count(a) AS attribute_count;
```

Table RAW

attribute_count
25

Started streaming 1 record after 33 ms and completed after 35 ms.

7. All attribute values of one selected object

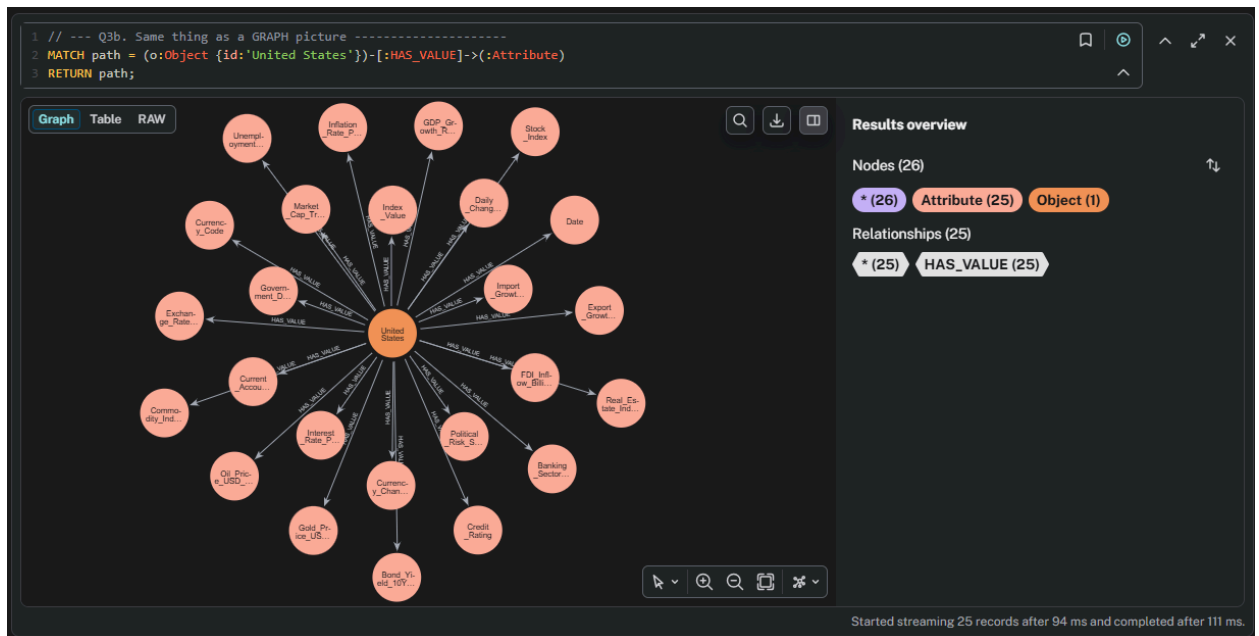
```
1 // --- Q3. All attribute values of one selected object -----
2 // pick any country as the object
3 MATCH (o:Object {id:'United States'})-[:HAS_VALUE]->(a:Attribute)
4 RETURN a.name AS attribute, r.value AS value
5 ORDER BY attribute;
```

Table RAW

attribute	value
"Banking_Sector_Health"	"Strong"
"Bond_Yield_10Y_Percent"	"4.25"
"Commodity_Index"	"1.12"
"Credit_Rating"	"AAA"
"Currency_Change_YTD_Percent"	"0.0"
"Currency_Code"	"USD"
"Current_Account_Balance_Billion_USD"	"-695.2"

Started streaming 25 records after 131 ms and completed after 156 ms.

8. Full graph

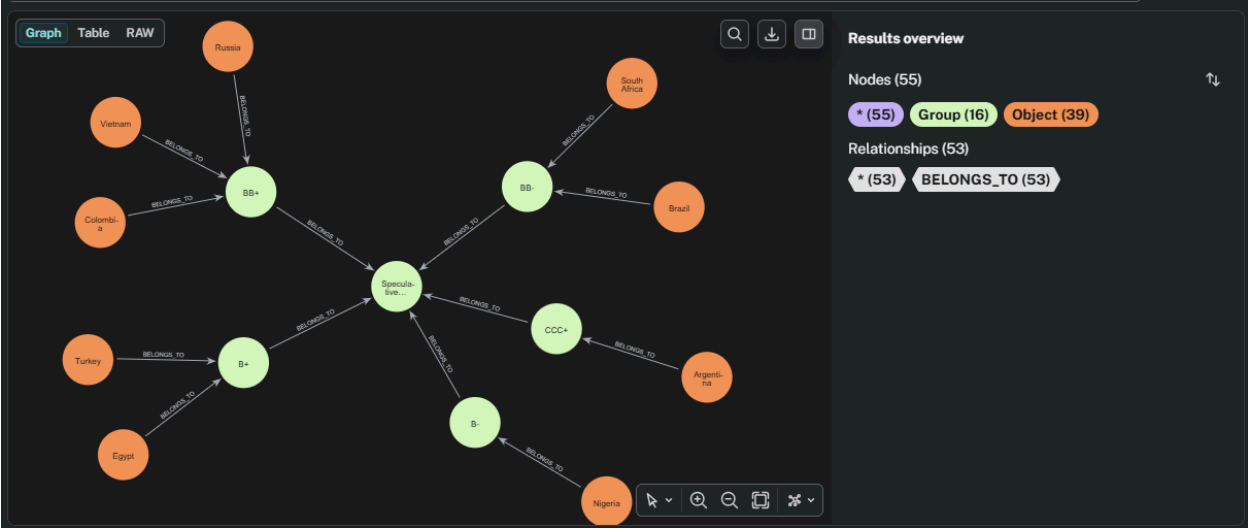


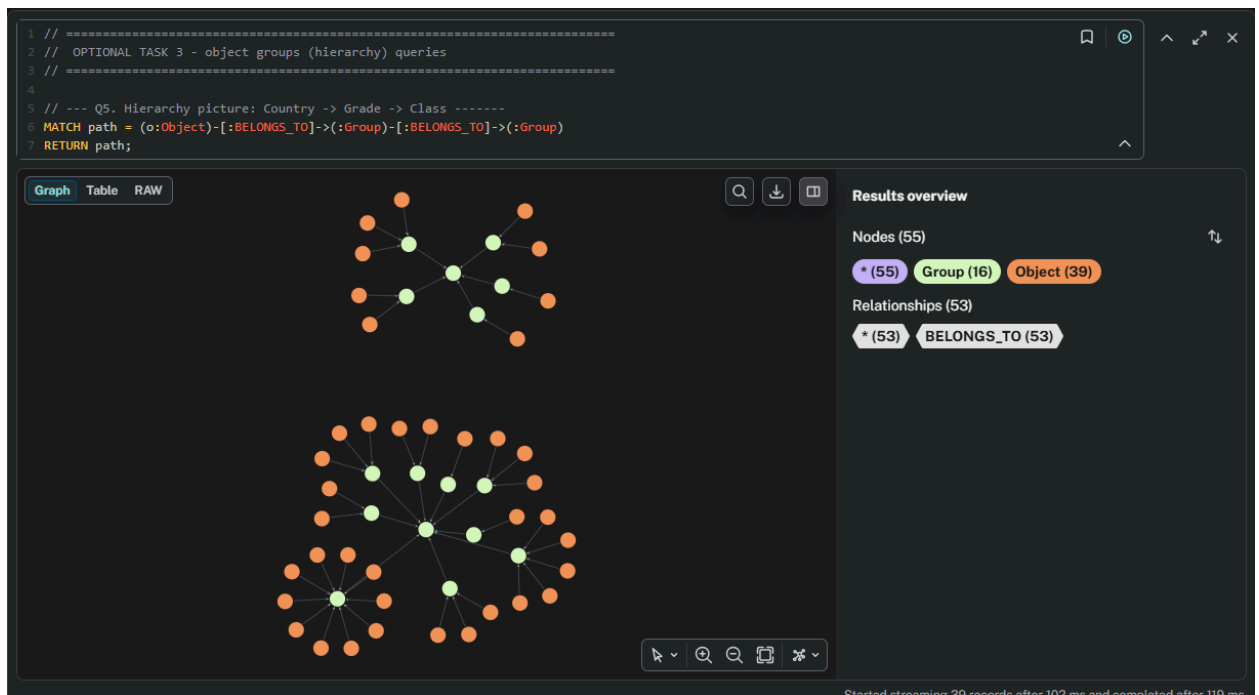
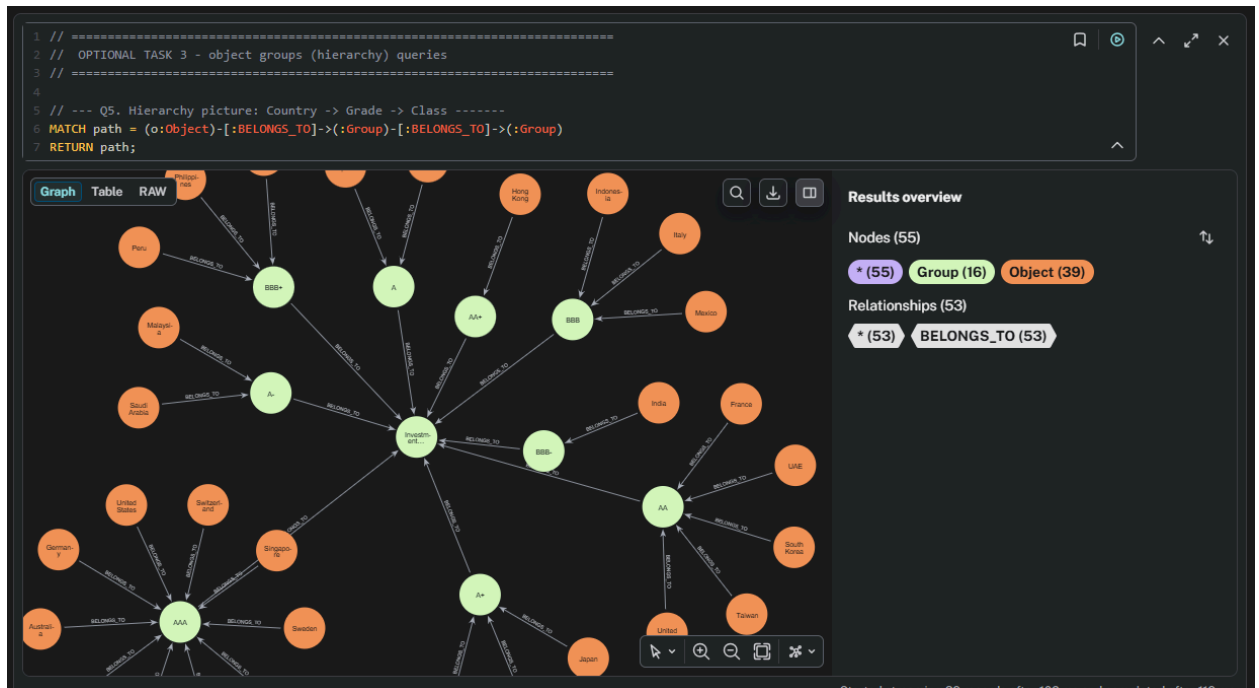
9. The query identifies and lists all countries that have a perfect 'AAA' credit rating and a GDP growth rate greater than 2.0%, returning their names, credit ratings, and GDP growth rates sorted from highest to lowest economic growth.



Optional tasks :

10. This query retrieves and visualizes the complete hierarchical path for each country, showing its specific credit rating grade and the broader credit rating class (Investment or Speculative) to which that grade belongs.





11. This query retrieves all countries that fall under the "Investment Grade" class, grouping and listing them by their specific credit rating grades in alphabetical order.

```
1 // --- Q6. Retrieve all objects in a top group via graph path-
2 //      all countries that belong (through their grade) to Investment Grade
3 MATCH (o:Object)-[:BELONGS_TO]->(grade:Group)-[:BELONGS_TO]->(cls:Group {name:'Investment Grade'})
4 RETURN cls.name AS class, grade.name AS grade, collect(o.id) AS countries
5 ORDER BY grade;
```

	class	grade	countries
1	"Investment Grade"	"A"	["Spain", "Chile"]
2	"Investment Grade"	"A+"	["China", "Japan", "Israel"]
3	"Investment Grade"	"A-"	["Malaysia", "Saudi Arabia"]
4	"Investment Grade"	"AA"	["United Kingdom", "France", "South Korea", "Taiwan", "UAE"]
5	"Investment Grade"	"AA+"	["Hong Kong"]

Started streaming 9 records after 201 ms and completed after 225 ms.

Summary

A Pawlak Information System in Neo4j using global financial data 2024, representing countries as objects and economic metrics as attributes. The scripts import this data, create a credit rating hierarchy, and run graph queries to filter countries by specific economic conditions and explore their groupings.